Chap 2

Arrays and Structures

海大資訊工程系
蔡東佐

Outline

- Arrays
- Structures and Union
- Polynomials
- Sparse Matrix
- Representation of Multidimensional Arrays

Arrays – The Abstract Data Type

- Many programmers view an array only as "a consecutive set of memory locations".
  連續的集合
- We begin our discussion by considering an array as an ADT.

ADT *Array* is

objects: A set of pairs <*index*, *value*> where for each value of *index* there is a value from the set item. *Index* is a finite ordered set of one or more dimensions, for example, {0,…, $n-1$}, for one dimension, {(0,0), (0,1), (0,2), (1,0), (1,1), (1,2), (2,0), (2,1), (2,2)}, for two dimensions, etc.

functions:

for all $A \in Array$, $i \in index$, $x \in item$, j, $size \in$ integer

Array Create(*j*, *list*)	::=	**return** an array of *j* dimensions where list is a *j*-tuple whose *i*th element is the size of the *i*th dimension. *Items* are undefined.
Item Retrieve(*A*,*i*)	::=	**if**($i \in index$) **return** the item associated with index value *i* in array *A* **else return** error
Array Store (*A*,*i*,*x*)	::=	**if**(*i* in *index*) **return** an arraythat is indetical to array A except the new pair <*i*, *x*> has been inserted. **else return** error

end *Array*

ADT 2.1：Abstract Data Type *Array*

Arrays – Arrays in C

- A one-dimensional array in C is declared implicitly by appending brackets to the name of a variable.
  - ✓ int list[5]; // five integers
  - ✓ int *plist[5]; // five pointers to integers
- When the complier encounters an array declaration such as "**int list[5]**", **it allocates five consecutive memory locations**.

Variable	Memory Address
list[0]	base address = **α**
list[1]	**α** + sizeof(int)
list[2]	**α** + 2 * sizeof(int)
list[3]	**α** + 3 * sizeof(int)
list[4]	**α** + 4 * sizeof(int)

The address of the first element list[0], is called the **base address**.

Arrays – Arrays in C (two cases)

- Observe that there is a difference between a declaration such as
  - ✓int *list1
  - ✓int list2[5]
- Same
  - ✓The variables list1 and list2 are both pointers to an int.
- Different
  - ✓In list2, five memory locations for holding integers have been reserved.

```
void print1 (int one[], int n) {
    for (int i=0; i<n; i++){
        cout << one[i] <<" "<<&one[i]<<"\n";
    }
}
```

Arrays – Arrays in C (Example)

- Assume that we have the following declaration:
  - ✓ int one[ ] = {0, 1, 2, 3, 4};

We would like to write a function that prints out both the address of the *i*th element of this array and the value found at this address.

```
void print1(int *ptr, int rows) /* 函數並未配置新記憶體,僅將陣列的位址傳入*/
{
    /* print out a one-dimensional array using a pointer */
    int i;
    printf("Address Contents\n");
    for (i = 0; i < rows; i++)
        printf("%8u%5d\n", ptr+i, *(ptr+i));
    printf("\n");                /* ptr+i  = &one[i]*/
}
```

Function call: print1(one, 5);

Address	Contents
1228	0
1230	1
1232	2
1234	3
1236	4

Structures and Union – Structures 1/2

- **Arrays** are collection of data of the **same type**.
  - ✓int one[ ] = {0, 1, 2, 3, 4};
- A **structure** is a **collection of data items**, where each item is identified as to its type and name.
  - ✓struct name {
    datatype1 var1;
    datatype2 var2;
    …
    };

Structures and Union – Structures 2/2

- For example, the following structure creates a variable whose name is person and that has three fields:
  - ✓ a name that is a character array
  - ✓ an integer value representing the age of the person
  - ✓ a float value representing the salary of the individual

```
struct {
        char name[10];
        int age;
        float salary;
} person;
```

```
strcpy(person.name, “james”);
person.age = 10;
person.salary = 350;
```

Create Structure Data Types

- By using the **typedef** statement
  - ✓ typedef int ntou_int;

    ntou_int a, b;

    typedef struct employee human_being;
- Declare variables：

  等同 struct employee person1, person2;
  - ✓ human_being person1, person2;

    strcpy(person1.name, “james”);

    person1.age = 10;

    person1.salary = 35000;

```
struct employee {
        char name[10];
        int age;
        float salary;
} ;
```

A Structure within a Structure

- Embed a structure within a structure

```
typedef struct {
    int month;
    int day;
    int year;
} date;

typedef struct {
    char name[10];
    int age;
    float salary;
    date dob;
} human_being;
```

```
human_being person1;
person1.dob.month=2;
person1.dob.day=11;
person1.dob.year=1944;
```

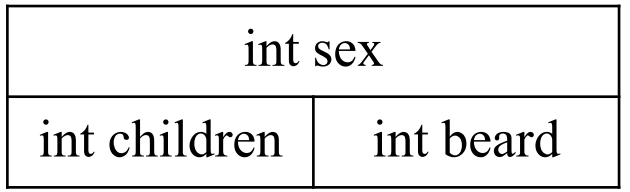
Union

能節省空間，需小心使用！

- Continuing with our humanBeing example, it would be nice if we could distinguish between males and females.
  - ✓Males: we might ask whether they have a beard or not.
  - ✓Females: we might wish to know the number of children they have borne.
- Union
  - ✓The fields of a **union** must **share their memory space**.
  - ✓Only one field of the **union** is "active" at any given time.

```
typedef struct {
   int sex;
   union {
      int children;
      int beard;   //留鬍子
   } u;
} sex_type;
```

```
sex_type person1;
person1.sex = 0;
person1.u.beard = 0;
```



Self-Referential Structures

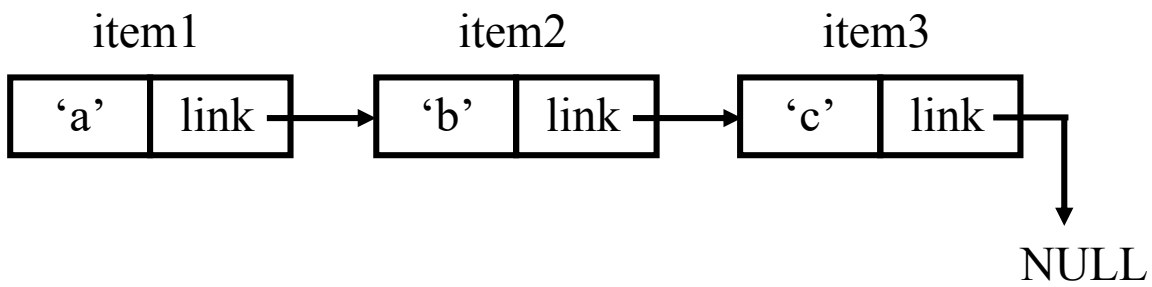
- One or more of its components is a pointer to itself.

```
struct node {
    char data;
    struct node *link;
} ;
typedef  struct node  list;
```

or

```
typedef  struct node {
    char data;
    struct node *link;
} list;
```

```
list item1, item2, item3;
item1.data='a';
item2.data='b';
item3.data='c';
item1.link=&item2; item2.link=&item3;
item3.link=NULL;
```



(Chapter 4)

Polynomials – Abstract Data Type 1/4

- Arrays are not only data structures in their own right, we can also use them to implement other abstract data types.
- Ordered or linear list:
  - ✓ Days of the week:
    - Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday
  - ✓ Values in a deck of card:
    - Ace, 2, 3, 4, 5, 6, 7, 8, 9, 10, Jack, Queen, King
  - ✓ Floors of a building:
    - basement, lobby, mezzanine , first, second
  - ✓ Years the United States fought in World War II:
    - 1941, 1942, 1943, 1944, 1945

Polynomials – Abstract Data Type 2/4

- Operations on Ordered List
  - ✓**Finding** the length, *n*, of the list.
  - ✓**Reading** the items from left to right (or right to left).
  - ✓**Retrieving** the *i*-th element.
  - ✓**Storing** a new value into the *i*-th position.
  - ✓**Inserting** a new element at the position *i* .

    (The items numbered ***i*, *i*+1** become numbered ***i*+1, *i*+2**….)
  - ✓**Deleting** the element at position *i*.

    (The items numbered ***i*+1, *i*+2** become numbered ***i*, *i*+1**….)

Polynomials – Abstract Data Type 3/4

- Represent as an array (a sequential mapping)
  - ✓Work well for (1) - (4)
  - ✓But, for inserting and deleting?

1. **Finding** the length, *n*, of the list.
2. **Reading** the items from left to right (or right to left).
3. **Retrieving** the *i*-th element.
4. **Storing** a new value into the *i*-th position.
5. **Inserting** a new element at the position *i* .
   (The items numbered ***i***, ***i*+1** become numbered ***i*+1**, ***i*+2**….)
6. **Deleting** the element at position *i*.
   (The items numbered ***i*+1**, ***i*+2** become numbered ***i***, ***i*+1**….)

Polynomials – Abstract Data Type 4/4

- To build a set of functions that allow for the manipulation of polynomials.
  - ✓ $A(x) = 3x^{20}+2x^5+4$ and $B(x) = x^4+10x^3+3x^2+1$
  - ✓ $A(x) + B(x) = ?$
  - ✓ $A(x) * B(x) = ?$

How to implement?

Polynomials – Representation 1/2

- Polynomial: $B(x) = x^4+10x^3+3x^2+1$
- Representation
  - ✓ Data structure 1:

```
#define MAX_DEGREE 101
typedef struct {
        int degree;
        float coef[MAX_DEGREE];
} polynomial;
```

polynomial a;
a.degree = n;
a.coef [i] = a_{n-i}, $0 \le i \le n$

$B(x) = x^4+10x^3+3x^2+1$
a.degree = 4
a.coef

1	10	3	0	1

advantage: easy implementation
disadvantage: waste space when sparse

```
#define MAX 101
```

前 v.s. 後

```
typedef struct{
    int degree;
    double coef[MAX];
} polynomial;
```

```
typedef struct{
    int degree;
    double num;
} polynomial_2[MAX];
```

Polynomials – Representation 2/2

A(x)使用六個儲存空間：startA、finishA、兩個存係數、兩個存次方

- Polynomial: $B(x) = x^4+10x^3+3x^2+1$ & $A(x)=2x^{1000}+1$

$A(x)=2x^{1000}+1$

$B(x)=x^4+10x^3+3x^2+1$

- Representation
  - ✓ Data structure 2:

```
#define MAX_TERMS 100
typedef  struct {
        float coef;
        int expon;
} polynomial;
polynomial  terms[MAX_TERMS];
int avail = 0;
```

specification	representation
poly	<start, finish>
A	<0,1>
B	<2,5>

	starta	*finisha*	*startb*			*finishb*	*avail*
coef	**2**	**1**	**1**	**10**	**3**	**1**	
exp	1000	0	4	3	2	0	

全部多項式的非零項的總個數不可以超過MAX_TERMS

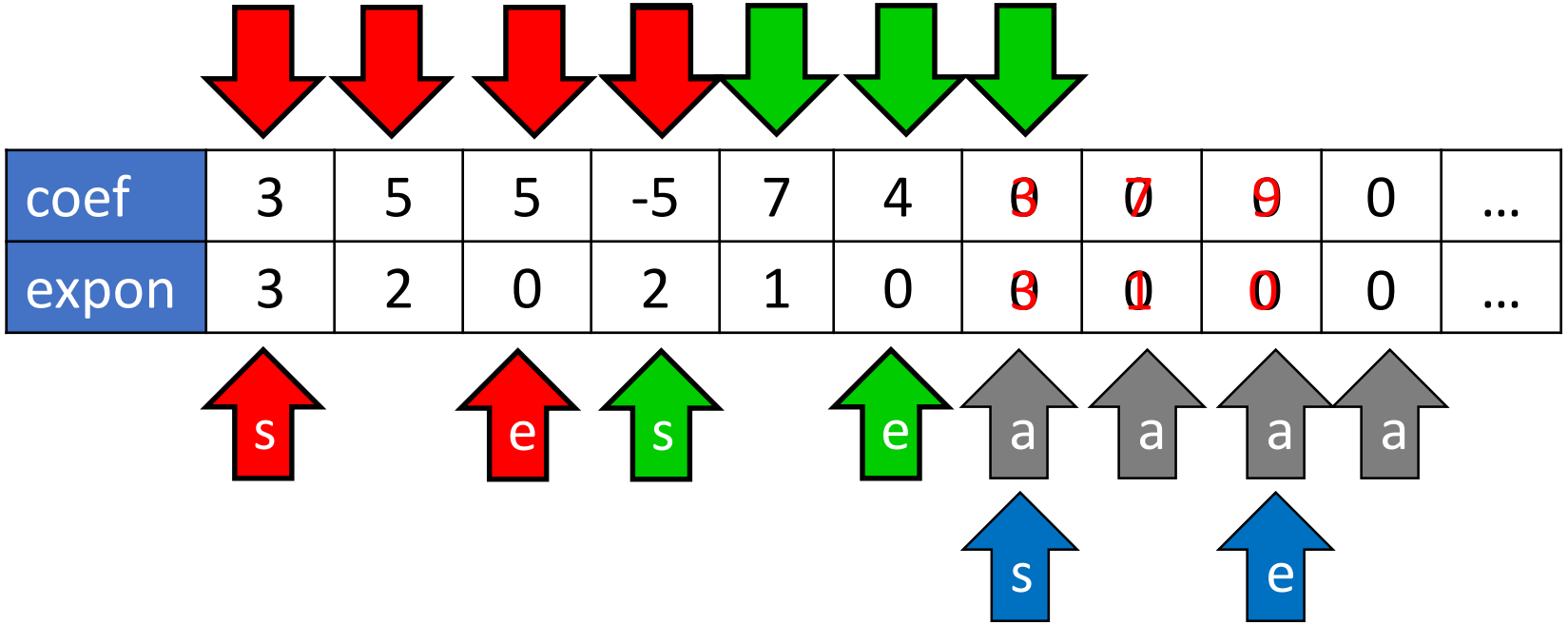
storage requirements: start, finish, 2*(finish-start+1)
nonsparse: twice as much as data structure 1 when all the items are nonzero

Polynomials – Addition 1/3

$3x^7 + 7x + 9$

➢void padd(int start_a,int end_a,int start_b,int end_b, int *start_d,int *end_d);

$3x^3+5x^2+5$ $-5x^2+7x+4$



Polynomials – Addition 2/3

➢ Program 2.6 Function to Add Two Polynomials

✓ void padd(int start_a,int end_a,int start_b,int end_b, int *start_d,int *end_d);

```
{
    /*add A(x) and B(x) to obtain D(x) */
    float coefficient;
    *startd = avail;
    while (starta <= finisha && startb <= finishb)
       switch(COMPARE(terms[starta].expon, terms[startb].expon)) {
          case -1: /* a expon < b expon */
                attach(terms[startb].coef, terms[startb].expon);
                startb++;
                break;
           case 0: /* equal expontents */
                coefficient = terms[starta]. coef + terms[startb].coef;
                if (coefficient) attach(coefficient, terms[starta].expon);
                starta++;
                startb++;
                break;
           case 1: /* a expon > b expon */
                attach(terms[starta].coef, terms[starta].expon);
                starta++;
    }
    /* add in remaining terms of A(x) */
    for (; starta <= finisha; starta++)
       attach(terms[starta].coef, terms[starta].expon);
    /* add in remaining terms of B(x) */
    for (; startb <= finishb; startb++)
       attach(terms[startb].coef, terms[startb].expon);
    *finishd = avail-1;
    }
```

可用空間

Analysis: *O*(n+m), where *n* (*m*) is the number of nonzeros in A(B).

Polynomials – Addition 3/3

- Program 2.7 Function to Add a New Term
  - ✓ void attach(float coefficient, int exponent)

```
{
    /* add a new term to the polynomial */
    if (avail > MAX_TERMS) {
        fprintf(stderr, "Too many terms in the polynomial\n");
        exit(1);
    }
    terms[avail].coef = coefficient;
    terms[avail++].expon = exponent;
}
```

Problem: Compaction is required when polynomials that are no longer needed.
(data movement takes time.)

Sparse Matrix

```
typedef struct {
    int r, c;
    int value;
} spm;
```

- As computer scientists, our interest centers not only on the specification of an appropriate ADT, but also on finding representations that let us efficiently perform the operations described in the specification.
- We write $m \times n$ to designate a matrix with ***m* rows** and ***n* columns**.
  - ✓ The total number of elements in such a matrix is mn.
  - ✓ If m equals n, the matrix is square.
- The standard representation of a matrix is a two dimensional array defined as

  a[MAX_ROWS][MAX_COLS]
  - ✓ We can locate quickly any element by writing $a[i][j]$.

	col0	col1	col2
row0	−27	3	4
row1	6	82	−2
row2	109	−64	11
row3	12	8	9
row4	48	27	47

15/15

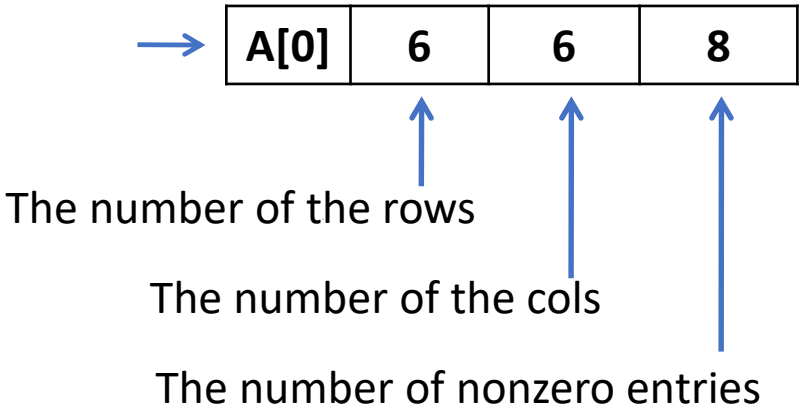
Sparse Matrix – the Abstract Data Type

- How to efficiently store the sparse matrix?
  - ✓We can locate quickly any element by writing $a[i][j]$?
- We consider alternate forms of representation as sparse matrix wastes space.
  - ✓Our representation of sparse matrices should **store only nonzero elements.**
  - ✓Each element is characterized uniquely by **<row, col, value>**.

	col0	col1	col2	col3	col4	col5
row0	15	0	0	22	0	−15
row1	0	11	3	0	0	0
row2	0	0	0	−6	0	0
row3	0	0	0	0	0	0
row4	91	0	0	0	0	0
row5	0	0	28	0	0	0

Sparse Matrix – Representation 1/2

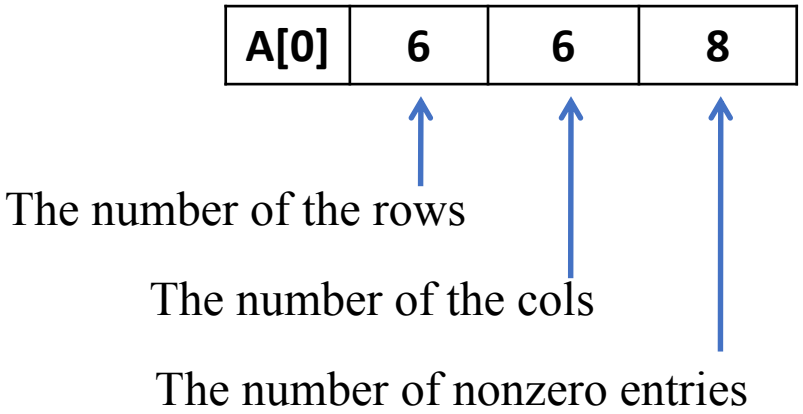
	col0	col1	col2	col3	col4	col5
row0	15	0	0	22	0	−15
row1	0	11	3	0	0	0
row2	0	0	0	−6	0	0
row3	0	0	0	0	0	0
row4	91	0	0	0	0	0
row5	0	0	28	0	0	0



	Row	Col	value
A[0]	6	6	8
A[1]	0	0	15
A[2]	0	3	22
A[3]	0	5	-15
A[4]	1	1	11
A[5]	1	2	3
A[6]	2	3	-6
A[7]	4	0	91
A[8]	5	2	28

Sparse Matrix – Representation 2/2

```
#define MAX_TERMS 101    /* maximum number of terms +1*/
   typedef struct {
         int col;
         int row;
         int value;
   } term;
    term a[MAX_TERMS]
```



	Row	Col	value
A[0]	6	6	8
A[1]	0	0	15
A[2]	0	3	22
A[3]	0	5	-15
A[4]	1	1	11
A[5]	1	2	3
A[6]	2	3	-6
A[7]	4	0	91
A[8]	5	2	28

Sparse Matrix – Transposing a Matrix 1/2

r, c 交換

	Row	Col	value
A[0]	6	6	8
A[1]	0	0	15
A[2]	0	3	22
A[3]	0	5	-15
A[4]	1	1	11
A[5]	1	2	3
A[6]	2	3	-6
A[7]	4	0	91
A[8]	5	2	28

	Row	Col	value
A[0]	6	6	8
A[1]	0	0	15
A[2]	0	4	91
A[3]	1	1	11
A[4]	2	1	3
A[5]	2	5	28
A[6]	3	0	22
A[7]	3	2	-6
A[8]	5	0	-15

row ↓ col →

$$\begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} 15 & 0 & 0 & 22 & 0 & -15 \\ 0 & 11 & 3 & 0 & 0 & 0 \\ 0 & 0 & 0 & -6 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 91 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 28 & 0 & 0 & 0 \end{bmatrix} \end{array}$$

row ↓ col →

$$\begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} 15 & 0 & 0 & 0 & 91 & 0 \\ 0 & 11 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 & 28 \\ 22 & 0 & -6 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ -15 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{array}$$

Sparse Matrix – Transposing a Matrix 2/2

- Algorithm 1:
  - ✓ for each row i
    - take element <i, j, value> and store it as element <j, i, value> of the transpose;
  - ✓ **difficulty: where to put <j, i, value>**
    - (0, 0, 15) ====> (0, 0, 15)
    - (0, 3, 22) ====> (3, 0, 22)
    - (0, 5, -15) ====> (5, 0, -15)
    - (1, 1, 11) ====> (1, 1, 11) ?

Move elements down very often

- Algorithm 2:
  - ✓ for all elements in column j
    - **place element <i, j, value> in element <j, i, value>**

	Row	Col	value
A[0]	6	6	8
A[1]	0	0	15
A[2]	0	3	22
A[3]	0	5	-15
A[4]	1	1	11
A[5]	1	2	3
A[6]	2	3	-6
A[7]	4	0	91
A[8]	5	2	28

	Row	Col	value
A[0]	6	6	8
A[1]	0	0	15
A[2]	0	4	91
A[3]	1	1	11
A[4]	2	1	3
A[5]	2	5	28
A[6]	3	0	22
A[7]	3	2	-6
A[8]	5	0	-15

Transposing a Matrix – Program 2.8 1/2

```
void transpose (term a[], term b[]) {
    int n, i, j, currentb;
    n=a[0].value;           /* total number of elements */
    b[0].row = a[0].col;    /* rows in b = columns in a     */
    b[0].col = a[0].row;    /* columns in b = rows in a     */
    b[0].value = n;
    if(n>0) {   /*non zero matrix */
        currentb=l;
        for(i=0; i<a[0].col; i++)
        /*transpose by the columns in a*/
            for (j=1; j<=n; j++)
             /* find elements from the current column*/
                if (a[j].col == i) {
                 /* element is in current column, add it to b */
                    b[currentb].row = a[j].col;
                    b[currentb].col = a[j].row;
                    b[currentb].value = a[j].value;
                    currentb++;
                 }
    }
}
```

	Row	Col	value
a[0]	6	6	8
a[1]	0	0	15
a[2]	0	3	22
a[3]	0	5	-15
a[4]	1	1	11
a[5]	1	2	3
a[6]	2	3	-6
a[7]	4	0	91
a[8]	5	2	28

	Row	Col	value
b[0]	6	6	8
b[1]	0	0	15
b[2]	0	4	91
b[3]	1	1	11
b[4]	2	1	3
b[5]	2	5	28
b[6]	3	0	22
b[7]	3	2	-6
b[8]	5	0	-15

Transposing a Matrix – Program 2.8 2/2

```
void transpose (term a[], term b[]) {
    int n, i, j, currentb;
    n=a[0].value;           /* total number of elements */
    b[0].row = a[0].col;    /* rows in b = columns in a     */
    b[0].col = a[0].row;    /* columns in b = rows in a     */
    b[0].value = n;
    if(n>0) {   /*non zero matrix */
        currentb=l;
        for(i=0; i<a[0].col; i++)
        /*transpose by the columns in a*/
            for (j=1; j<=n; j++)
             /* find elements from the current column*/
                if (a[j].col == i) {
                 /* element is in current column, add it to b */
                    b[currentb].row = a[j].col;
                    b[currentb].col = a[j].row;
                    b[currentb].value = a[j].value;
                    currentb++;
                }
    }
}
```

- Asymptotic time complexity of the transpose algorithm
  - ✓If the matrices is represented as 2-D arrays, the time complexity is
    - O(columns*rows)
  - ✓The time complexity of the transpose algorithm is
    - O(columns*elements) = O(columns*columns*rows)
    - Problem: **Scan the array "columns" times**.

Transposing a Matrix – Alternative solution

会考!

- Determine the number of elements in each column of the original matrix.
- Determine the starting positions of each row in the transpose matrix.

$$\begin{bmatrix} 15 & 0 & 0 & 22 & 0 & -15 \\ 0 & 11 & 3 & 0 & 0 & 0 \\ 0 & 0 & 0 & -6 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 91 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 28 & 0 & 0 & 0 \end{bmatrix} \Rightarrow \begin{bmatrix} 15 & 0 & 0 & 0 & 91 & 0 \\ 0 & 11 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 & 28 \\ 22 & 0 & -6 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ -15 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

	[0]	[1]	[2]	[3]	[4]	[5]
~~columns~~ row_terms	2	1	2	2	0	1
starting_pos	1	3	4	6	8	8

非0项个数

	Row	Col	value
a[0]	6	6	8
a[1]	0	0	15
a[2]	0	3	22
a[3]	0	5	-15
a[4]	1	1	11
a[5]	1	2	3
a[6]	2	3	-6
a[7]	4	0	91
a[8]	5	2	28

	Row	Col	value
b[0]	6	6	8
b[1]	0	0	15
b[2]	0	4	91
b[3]	1	1	11
b[4]	2	1	3
b[5]	2	5	28
b[6]	3	0	22
b[7]	3	2	-6
b[8]	5	0	-15

Transposing a Matrix – Program 2.9 1/3

會考!

```
void fast_transpose(term a[], term b[])
{
   int row_terms[MAX_COL], starting_pos[MAX_COL];
   int i, j, num_cols = a[0].col, num_terms = a[0].value;
   b[0].row=num_cols; b[0].col=a[0].row;
   b[0].value =num_terms;
   if (num_terms > 0) { /* nonzero matrix */
      for(i=0; i< num_cols; i++)
         row_terms[i]=0;
      for(i=1; i<=num_terms; i++)
         row_terms[a[i].col]++;
      starting_pos[0]=1;
      for(i=1; i< num_cols; i++)
         starting_pos[i] = starting_pos[i-1]+row_terms[i-1];
      for(i=1; i<=num_terms; i++) {
         j=starting_pos[a[i].col]++;
         b[j].row=a[i].col; b[j].col=a[i].row;
         b[j].value=a[i].value;
      }
   }
}
```

columns

elements

columns

elements 會考!

$$\begin{bmatrix} 15 & 0 & 0 & 22 & 0 & -15 \\ 0 & 11 & 3 & 0 & 0 & 0 \\ 0 & 0 & 0 & -6 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 91 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 28 & 0 & 0 & 0 \end{bmatrix} \Rightarrow \begin{bmatrix} 15 & 0 & 0 & 0 & 91 & 0 \\ 0 & 11 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 & 28 \\ 22 & 0 & -6 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ -15 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

	[0]	[1]	[2]	[3]	[4]	[5]
row_terms	2	1	2	2	0	1
starting_pos	1	3	4	6	8	8

	Row	Col	value
a[0]	6	6	8
a[1]	0	0	15
a[2]	0	3	22
a[3]	0	5	-15
a[4]	1	1	11
a[5]	1	2	3
a[6]	2	3	-6
a[7]	4	0	91
a[8]	5	2	28

⇨

	Row	Col	value
b[0]	6	6	8
b[1]	0	0	15
b[2]	0	4	91
b[3]	1	1	11
b[4]	2	1	3
b[5]	2	5	28
b[6]	3	0	22
b[7]	3	2	-6
b[8]	5	0	-15

注意：
j=starting_pos[a[i].col]++;
表示
j=starting_pos[a[i].col]然後
starting_pos[a[i].col]++

	Row	Col	value
a[0]	6	6	8
a[1]	0	0	15
a[2]	0	3	22
a[3]	0	5	-15
a[4]	1	1	11
a[5]	1	2	3
a[6]	2	3	-6
a[7]	4	0	91
a[8]	5	2	28

i	j=starting_pos[a[i].col]++;
1	j=1, then starting_pos[0]=2
2	j=6, then starting_pos[3]=7
3	j=8, then starting_pos[5]=9
4	j=3, then starting_pos[1]=4
5	j=4, then starting_pos[2]=5
6	j=7, then starting_pos[3]=8
7	j=2, then starting_pos[0]=3
8	j=5, then starting_pos[2]=6

Transposing a Matrix – Program 2.9 2/3

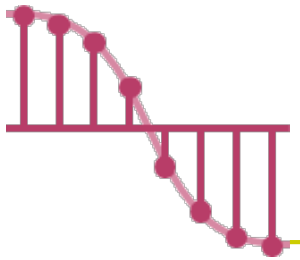
	Row	Col	value
a[0]	6	6	8
a[1]	0	0	15
a[2]	0	3	22
a[3]	0	5	-15
a[4]	1	1	11
a[5]	1	2	3
a[6]	2	3	-6
a[7]	4	0	91
a[8]	5	2	28

i	j=starting_pos[a[i].col]++;
1	j=1, then starting_pos[0]=2
2	j=6, then starting_pos[3]=7
3	j=8, then starting_pos[5]=9
4	j=3, then starting_pos[1]=4
5	j=4, then starting_pos[2]=5
6	j=7, then starting_pos[3]=8
7	j=2, then starting_pos[0]=3
8	j=5, then starting_pos[2]=6

```
b[j].row=a[i].col;
b[j].col=a[i].row;
b[j].value=a[i].value;
```

	Row	Col	value
b[0]	6	6	8
b[1]	0	0	15
b[2]	0	4	91
b[3]	1	1	11
b[4]	2	1	3
b[5]	2	5	28
b[6]	3	0	22
b[7]	3	2	-6
b[8]	5	0	-15

$$\begin{bmatrix} 15 & 0 & 0 & 22 & 0 & -15 \\ 0 & 11 & 3 & 0 & 0 & 0 \\ 0 & 0 & 0 & -6 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 91 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 28 & 0 & 0 & 0 \end{bmatrix} \Rightarrow \begin{bmatrix} 15 & 0 & 0 & 0 & 91 & 0 \\ 0 & 11 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 & 28 \\ 22 & 0 & -6 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ -15 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$



Transposing a Matrix – Program 2.9 3/3

- Asymptotic time complexity of the fast transpose algorithm
  - ✓The transpose algorithm is
    - O(columns*elements) = O(columns*columns*rows)
  - ✓The fast transpose algorithm is
    - O(columns + elements) = O(columns + columns*rows)
    - **Cost :** Additional **row_terms** and **starting_pos** arrays are required

Matrix Multiplication

実習 会寫這个

- **Definition:**
  - ✓Given A and B where A is $m \times n$ and B is $n \times p$, the product matrix D has dimension $m \times p$. Its $<i, j>$ element is

$$d_{ij} = \sum_{k=0}^{n-1} a_{ik} b_{kj}$$

for $0 \le i < m$ and $0 \le j < p$

- For example

$$\begin{bmatrix} 1 & 0 & -2 \\ 0 & 3 & -1 \end{bmatrix} \begin{bmatrix} 0 & 3 \\ -2 & -1 \\ 0 & 4 \end{bmatrix} = \begin{bmatrix} 0 & -5 \\ -6 & -7 \end{bmatrix}$$

Sparse Matrix Multiplication 1/4

$$A = \begin{bmatrix} -27 & 6 \\ 82 & 0 \\ 109 & -64 \end{bmatrix} \qquad B = \begin{bmatrix} 21 & 27 & 0 \\ 5 & 0 & -71 \end{bmatrix} \qquad B^T = \begin{bmatrix} 21 & 5 \\ 27 & 0 \\ 0 & -71 \end{bmatrix}$$

	Row	Col	value
A[0]	3	2	5
A[1]	0	0	-27
A[2]	0	1	6
A[3]	1	0	82
A[4]	2	0	109
A[5]	2	1	-64
A[6]	3		

	Row	Col	value
B[0]	2	3	4
B[1]	0	0	21
B[2]	0	1	27
B[3]	1	0	5
B[4]	1	2	-71

	Row	Col	value
B^T[0]	3	2	4
B^T[1]	0	0	21
B^T[2]	0	1	5
B^T[3]	1	0	27
B^T[4]	2	1	-71
B^T[5]	3	-1	

boundary condition

設定邊界條件為第0列(row)的值

Sparse Matrix Multiplication 2/4

- By the definition: $[D]_{m*p}=[A]_{m*n}* [B]_{n*p}$
  - ✓Procedure: **Fix a row** of A and **find all elements in column** j of B for j =0, 1, …, p-1.
- Method 1 : Scan all of B to find all elements in j.
- Method 2 : Compute the transpose of B.

(Put all column elements consecutively)

Sparse Matrix Multiplication 3/4

$$A \times B = \begin{bmatrix} d_{00} & d_{01} & d_{02} \\ d_{10} & d_{11} & d_{12} \\ d_{20} & d_{21} & d_{22} \end{bmatrix}$$

$$A = \begin{bmatrix} -27 & 6 \\ 82 & 0 \\ 109 & -64 \end{bmatrix} \qquad B^T = \begin{bmatrix} 21 & 5 \\ 27 & 0 \\ 0 & -71 \end{bmatrix}$$

$$A = \begin{bmatrix} -27 & 6 \\ 82 & 0 \\ 109 & -64 \end{bmatrix} \qquad B = \begin{bmatrix} 21 & 27 & 0 \\ 5 & 0 & -71 \end{bmatrix}$$

	Row	Col	value
A[0]	3	2	5
A[1]	0	0	-27
A[2]	0	1	6
A[3]	1	0	82
A[4]	2	0	109
A[5]	2	1	-64
A[6]	3		

	Row	Col	value
B^T[0]	3	2	4
B^T[1]	0	0	21
B^T[2]	0	1	5
B^T[3]	1	0	27
B^T[4]	2	1	-71
B^T[5]	3	-1	

$$d_{00} = -27 \times 21 + 6 \times 5$$
$$d_{01} = -27 \times 27 + 6 \times 0$$
$$d_{02} = -27 \times 0 + 6 \times -71$$

A x B =

-537 -729 -426
1722 2214 0
1969 2943 4544

$$d_{10} = 82 \times 21 + 0 \times 5$$
$$d_{11} = 82 \times 27 + 0 \times 0$$
$$d_{12} = 82 \times 0 + 0 \times -71$$

$$d_{20} = 109 \times 21 + (-64) \times 5$$
$$d_{21} = 109 \times 27 + (-64) \times 0$$
$$d_{22} = 109 \times 0 + (-64) \times (-71)$$

Sparse Matrix Multiplication 4/4

[illegible] 的 column

$$A \times B = \begin{bmatrix} d_{00} & d_{01} & d_{02} \\ d_{10} & d_{11} & d_{12} \\ d_{20} & d_{21} & d_{22} \end{bmatrix}$$

$$A = \begin{bmatrix} -27 & 6 \\ 82 & 0 \\ 109 & -64 \end{bmatrix} \quad B = \begin{bmatrix} 21 & 27 & 0 \\ 5 & 0 & -71 \end{bmatrix}$$

$$A = \begin{bmatrix} -27 & 6 \\ 82 & 0 \\ 109 & -64 \end{bmatrix} \quad B^T = \begin{bmatrix} 21 & 5 \\ 27 & 0 \\ 0 & -71 \end{bmatrix}$$

$$d_{00} = -27 \times 21 + 6 \times 5$$

$$d_{01} = -27 \times 27 + 6 \times 0$$

$$d_{02} = -27 \times 0 + 6 \times -71$$

	Row	Col	value
A[0]	3	2	5
A[1]	0	0	-27
A[2]	0	1	6
A[3]	1	0	82
A[4]	2	0	109
A[5]	2	1	-64
A[6]	3		

	Row	Col	value
B^T[0]	3	2	4
B^T[1]	0	0	21
B^T[2]	0	1	5
B^T[3]	1	0	27
B^T[4]	2	1	-71
B^T[5]	3	-1	

sum = $-27 \times 21 + 6 \times 5$

row => 0

column => [illegible]

row_begin => 1

i = [illegible]

j = [illegible]

如果B^T[3].row = 0, 就下移.

Sparse Matrix Multiplication – Program 2.10 1/2

```
void mmult (term a[ ], term b[ ], term d[ ] )
/* multiply two sparse matrices */
{
  int i, j, column, totalb = b[0].value, totald = 0;
  int rows_a = a[0].row, cols_a = a[0].col,
  totala = a[0].value; int cols_b = b[0].col,
  int row_begin = 1, row = a[1].row, sum =0;
  term new_b[MAX_TERMS];
  if (cols_a != b[0].row){
       fprintf (stderr, "Incompatible matrices\n");
       exit (1);
  }
```

```
fast_transpose(b, new_b);
 /* set boundary condition */
 a[totala+1].row = rows_a;
 new_b[totalb+1].row = cols_b;
 new_b[totalb+1].col = -1;
 for (i = 1; i <= totala; ) {
     column = new_b[1].row;
     for (j = 1; j <= totalb+1;) {
     /* mutiply row of a by column of b */
         if (a[i].row != row)  {
           storesum(d, &totald, row, column, &sum);
           i = row_begin;
           for (; new_b[j].row == column; j++)
             ;
           column =new_b[j].row
         }
```

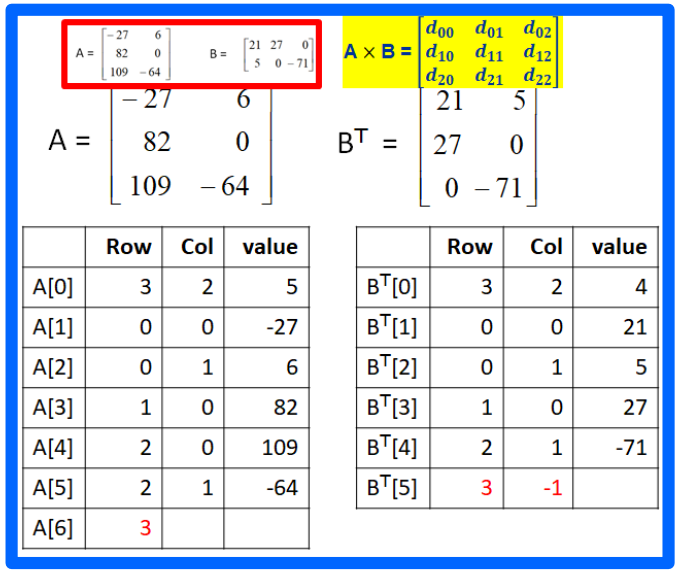
$A = \begin{bmatrix} -27 & 6 \\ 82 & 0 \\ 109 & -64 \end{bmatrix}$ $B^T = \begin{bmatrix} 21 & 5 \\ 27 & 0 \\ 0 & -71 \end{bmatrix}$ $A \times B = \begin{bmatrix} d_{00} & d_{01} & d_{02} \\ d_{10} & d_{11} & d_{12} \\ d_{20} & d_{21} & d_{22} \end{bmatrix}$

	Row	Col	value
A[0]	3	2	5
A[1]	0	0	-27
A[2]	0	1	6
A[3]	1	0	82
A[4]	2	0	109
A[5]	2	1	-64
A[6]	3		

	Row	Col	value
B^T[0]	3	2	4
B^T[1]	0	0	21
B^T[2]	0	1	5
B^T[3]	1	0	27
B^T[4]	2	1	-71
B^T[5]	3	-1	

Sparse Matrix Multiplication – Program 2.10 2/2

```
else if (new_b[j].row != column)  {
            storesum(d, &totald, row, column, &sum);
            i = row_begin;
            column = new_b[j].row;
          }
          else switch (COMPARE (a[i].col, new_b[j].col)) {
            case -1: /* go to next term in a */
                     i++; break;
            case 0: /* add terms, go to next term in a and b */
                     sum += (a[i++].value * new_b[j++].value);
                     break;
            case 1: /* advance to next term in b*/
                     j++
          }
      } /* end of for j <= totalb+1 */
      for (; a[i].row == row; i++)
          ;
      row_begin = i; row = a[i].row;
   } /* end of for i <=totala */
   d[0].row = rows_a;
   d[0].col = cols_b; d[0].value = totald;
}
```

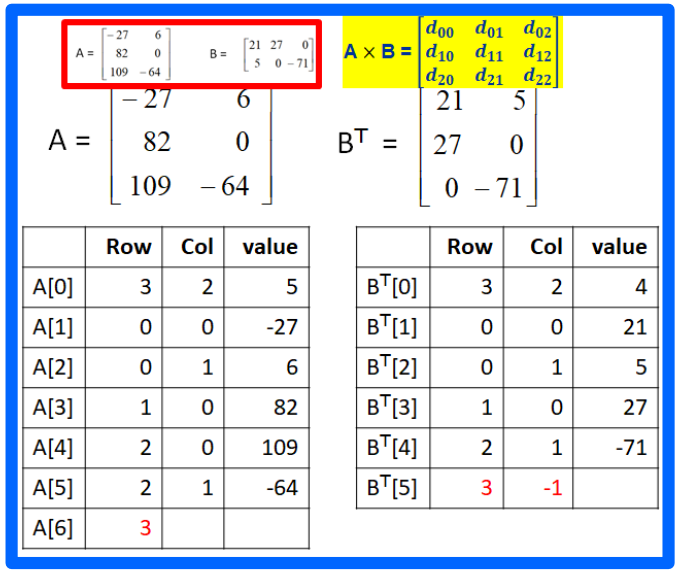


	Row	Col	value
A[0]	3	2	5
A[1]	0	0	-27
A[2]	0	1	6
A[3]	1	0	82
A[4]	2	0	109
A[5]	2	1	-64
A[6]	3		

	Row	Col	value
B^T[0]	3	2	4
B^T[1]	0	0	21
B^T[2]	0	1	5
B^T[3]	1	0	27
B^T[4]	2	1	-71
B^T[5]	3	-1	

Sparse Matrix Multiplication – Program 2.11

```
void storesum(term d[ ], int *totald, int row, int column, int *sum)
{
 /* if *sum != 0, then it along with its row and column
    position is stored as the *totald+1 entry in d */
  if  (*sum)
    if (*totald < MAX_TERMS) {
       d[++*totald].row = row;
       d[*totald].col = column;
       d[*totald].value = *sum;
    }
    else {
       printf("Numbers of terms in product exceed %d\n",
MAX_TERMS);
       exit(1);
    }
}
```



$$A = \begin{bmatrix} -27 & 6 \\ 82 & 0 \\ 109 & -64 \end{bmatrix} \qquad B^T = \begin{bmatrix} 21 & 5 \\ 27 & 0 \\ 0 & -71 \end{bmatrix}$$

$$A \times B = \begin{bmatrix} d_{00} & d_{01} & d_{02} \\ d_{10} & d_{11} & d_{12} \\ d_{20} & d_{21} & d_{22} \end{bmatrix}$$

	Row	Col	value
A[0]	3	2	5
A[1]	0	0	-27
A[2]	0	1	6
A[3]	1	0	82
A[4]	2	0	109
A[5]	2	1	-64
A[6]	3		

	Row	Col	value
B^T[0]	3	2	4
B^T[1]	0	0	21
B^T[2]	0	1	5
B^T[3]	1	0	27
B^T[4]	2	1	-71
B^T[5]	3	-1	

Sparse Matrix Multiplication – Asymptotic Time Complexity 1/2

- **cols_b : the number of columns in matrix B.**
  rows_a: the number of rows in matrix A.
  $termsrow_i$: the number of terms in row i of A.
- totalb : the number of elements in matrix B.

cols_b * $termsrow_0$ + totalb +
cols_b * $termsrow_1$ + totalb +
… +
cols_b * $termsrow_p$ + totalb

= cols_b * ($termsrow_1$ + $termsrow_2$ + … + $termsrow_p$) + rows_a * totalb

= **cols_b * totala + row_a * totalb**

Time complexity: O(cols_b * totala + rows_a * totalb + cols_b + totalb)

= O(cols_b * totala + rows_a * totalb)

Fast transpose algorithm

Sparse Matrix Multiplication – Asymptotic Time Complexity 2/2

- The classic multiplication algorithm

```
for (i =0; i < rows_a; i++)
    for (j=0; j < cols_b; j++) {
        sum =0;
        for (k=0; k < cols_a; k++)
            sum += (a[i][k] *b[k][j]);
        d[i][j] =sum;
    }
```

- The time complexity is O(rows_a * cols_b * cols_a)
- Program 2.10 : O(cols_b * total_a + rows_a * total_b)
  - ✓ **optimal case: total_a < rows_a * cols_a and total_b < cols_a * cols_b**
  - ✓ **worse case: total_a > rows_a * cols_a or total_b > cols_a * cols_b**

Representation of Multidimensional Arrays

- Multidimensional arrays are usually implemented by one dimensional array.
- There are two common ways to represent multidimensional arrays
  - ✓**Row major** and **column major**
  - ✓Only consider row major here
- For example, A[3][2] is implemented as

α	α + 1	α + 2	α + 3	α + 4	α + 5
A[0][0]	A[0][1]	A[1][0]	A[1][1]	A[2][0]	A[2][1]

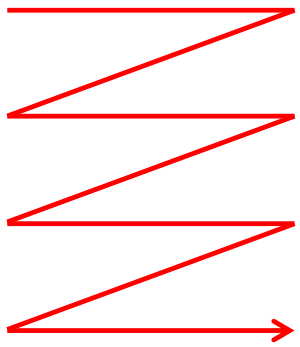
Two-Dimensional Arrays

- **Row major order stores two-dimensional arrays by rows.**
  - ✓ $A[u_0][u_1]$ is interpreted as u_0 rows, row_0, row_1, …, row_{u0-1}, each row containing u_1 elements.
- **$\alpha + i * u_1 + j$**, where α is the address of A[0][0].

α α+1 …… α+u_1-1

	col_0	col_1		col_{u1-1}
row_0	A[0][0]	A[0][1]	. . .	A[0][u_1-1]
row_1	A[1][0]	A[1][1]	. . .	A[1][u_1-1]
	. . .	. . .	. . .	. . .
row_{u0-1}	A[u_0-1][0]	A[u_0-1][1]		A[u_0-1][u_1-1]

α+u_0u_1-1



會考！
多維陣列(三維)的mem陣列
地址表示方式

Three-Dimensional Arrays

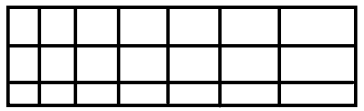
- $A[u_0][u_1][u_2]$ is interpreted as u_0 two-dimensional arrays of dimension $u_1 \times u_2$.
  - ✓ $\alpha + i*u_1*u_2$ as there are i two dimensional arrays of size u_1*u_2 preceding this element.
  - ✓ The address of a[i][j][k] is $\alpha + i*u_1*u_2 + j*u_2 + k$.

Let's Pratice:

A[7]

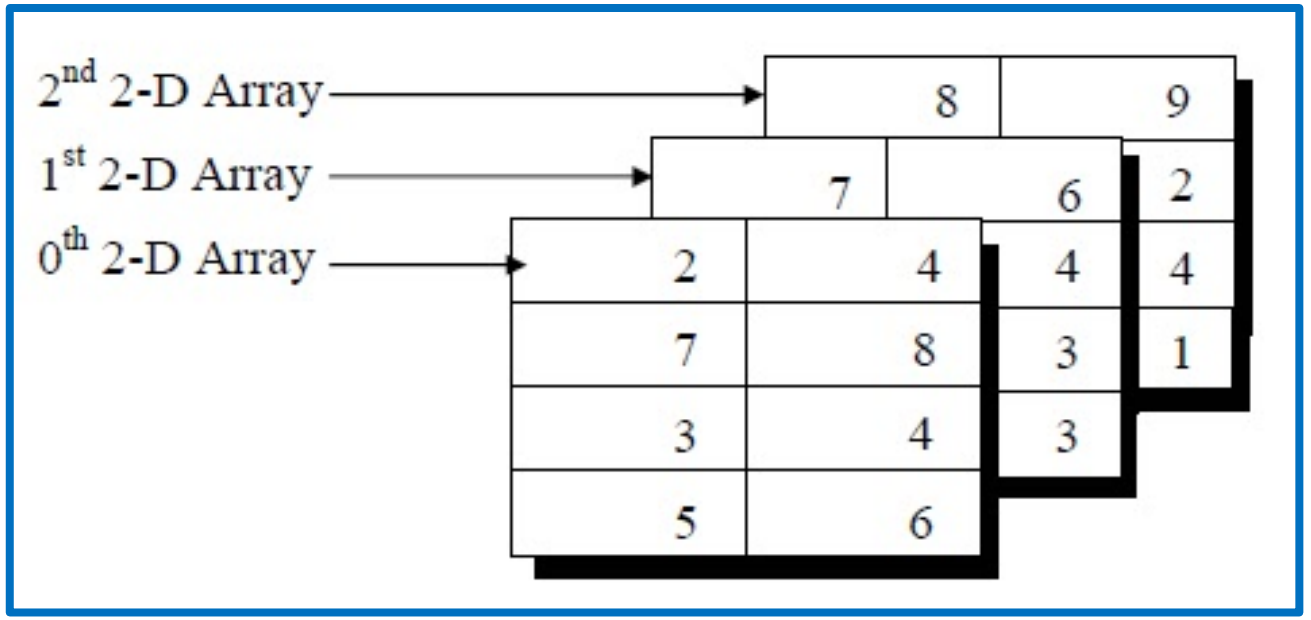
A[0] = α A[5] = α+5

B[3][7]



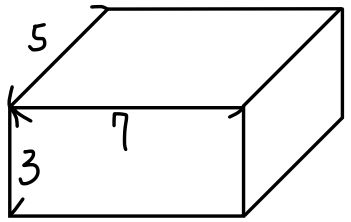
B[0][0] = α

B[1][5] = α+1×7+5



Let's Pratice

C[5][3][7]



C[0][0][0] = α

C[4][2][3] = α+4×3×7+2×7+3

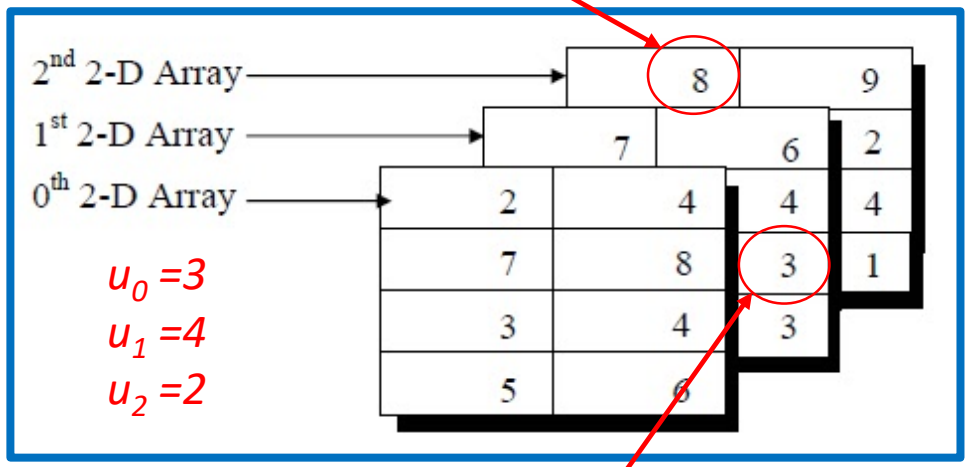
Multidimensional Arrays

- The address of A[i_0][i_1],…,[i_{n-1}] is :

$$\begin{aligned}
&\alpha + i_0 u_1 u_2 \ldots u_{n-1} \\
&\quad + i_1 u_2 u_3 \ldots u_{n-1} \\
&\quad + i_2 u_3 u_4 \ldots u_{n-1} \\
&\quad \vdots \\
&\quad + i_{n-2} u_{n-1} \\
&\quad + i_{n-1}
\end{aligned}$$

$$= \alpha + \sum_{j=0}^{n-1} i_j a_j \quad \text{where} \quad \begin{cases} a_j = \prod_{k=j+1}^{n-1} u_k & 0 \le j < n-1 \\ a_{n-1} = 1 \end{cases}$$

address of A[2][0][0] = α + 2*4*2+ 0*2 + 0= α + 16



address of A[1][2][1] = α + 1*4*2+ 2*2 + 1= α + 13